



Vue.js 라이프사이클 **혹**

생성 · 마운트 · 업데이트 · 언마운트의 흐름

- ✓ 라이프사이클의 4단계 흐름 완벽 이해
- ✓ 실무 필수 **혹**: onMounted, onUpdated, onUnmounted
- ✓ 타이머 컴포넌트 예제로 배우는 Clean-up 패턴

“ 라이프사이클 훅이란?

컴포넌트가 생성되고, 화면에 부착되고, 업데이트되고, 결국 소멸될 때까지의 특정 시점에 자동으로 호출되는 특별한 함수들을 말합니다.

01

라이프사이클의 큰 흐름 이해하기

생성 → 마운트 → 업데이트 → 언마운트의 4단계 프로세스 파악



02

필수 훅 3가지 정복 onMounted / Updated / Unmounted

Vue 3 Composition API에서 가장 자주 사용되는 핵심 훅 활용법



03

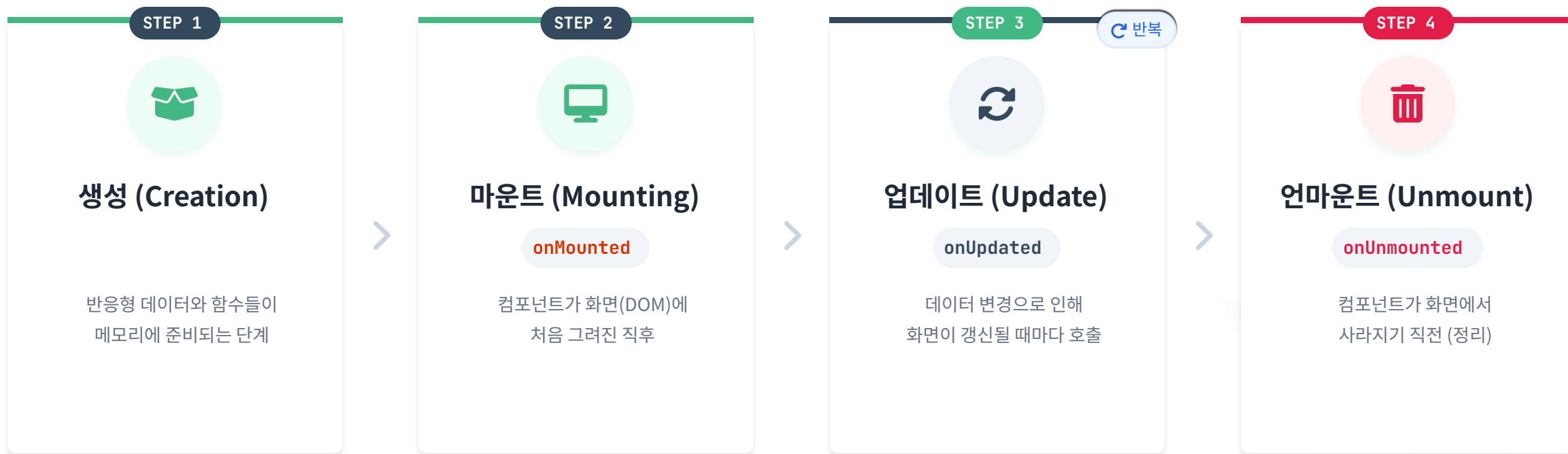
실전 예제: 타이머 컴포넌트

타이머를 만들며 생성·업데이트·자원 정리(Clean-up) 직접 체험



라이프사이클 개념과 흐름

컴포넌트의 일생: 생성부터 소멸까지 4단계 프로세스



<https://ko.vuejs.org/guide/essentials/lifecycle>

실무에서 자주 쓰는 3가지 훅

라이프사이클 훅의 종류는 많지만, 실제 개발에서는 주로 다음 3가지를 사용하여 대부분의 로직을 처리합니다.



onMounted

초기화

컴포넌트가 화면에 처음 나타났을 때 (1회 실행)

- API 데이터 호출 (Data Fetching)
- DOM 요소에 직접 접근해야 하는 작업
- 초기화 로직 실행



onUpdated

반응형 갱신

반응형 데이터가 변경되어 화면이 다시 그려진 뒤

- 변경된 데이터에 따른 추가 DOM 조작
- 차트나 외부 라이브러리 화면 갱신
- 디버깅용 변경 로그 출력



onUnmounted

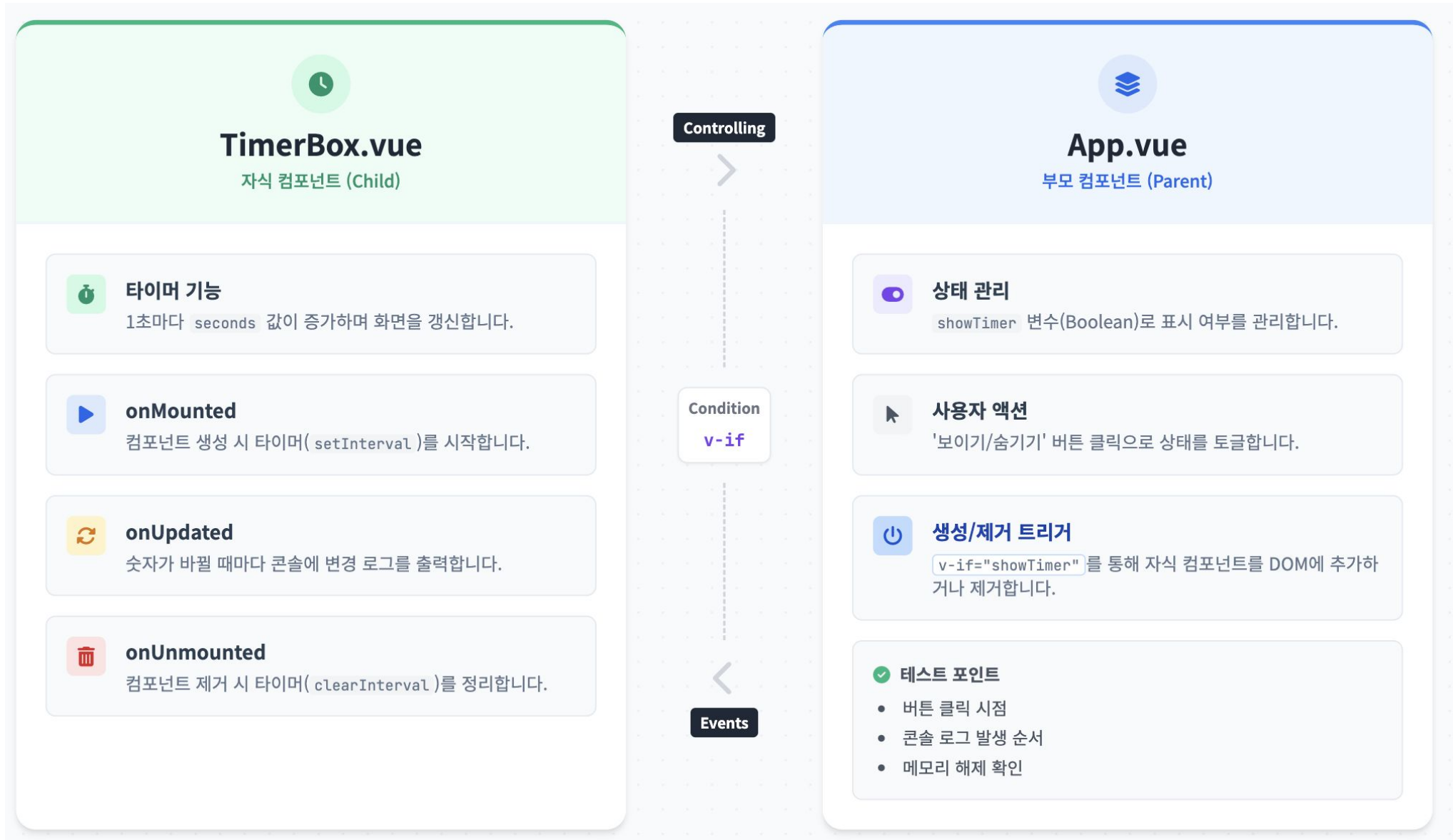
정리 (Clean-up)

컴포넌트가 화면에서 제거되기 직전

- setInterval, setTimeout 등 타이머 해제
- window/document에 등록한 이벤트 리스너 제거
- 웹소켓 연결 종료 등 메모리 누수 방지

실습 구조: TimerBox & App

부모-자식 컴포넌트 간의 상호작용과 라이프사이클 트리거



onMounted | 타이머 시작

src/components/TimerBox.vue

```
// 컴포넌트가 화면에 처음 나타날 때 실행

onMounted(() => {
  console.log('TimerBox onMounted: 타이머 시작' )

  // 1초마다 seconds 값을 1씩 증가
  intervalId = window.setInterval(() => {
    seconds.value ++
  }, 1000)
})
```

KEY CONCEPT

초기화 작업의 최적 시점

컴포넌트가 DOM에 완전히 부착된 직후 호출됩니다.

이때부터 DOM 요소 접근이나 외부 데이터 가져오기(Fetch API)가 안전하게 가능합니다.

LOGIC FLOW

setInterval 활용

브라우저 API인 setInterval을 사용해 주기적인 작업을 예약합니다.

반응형 상태(seconds) 변경

ID 저장 (나중에 멈추기 위해 필수!)

주의할 점

이곳에서 시작한 타이머나 이벤트 리스너는 반드시 onUnmounted에서 정리해 주어야 메모리 누수를 막을 수 있습니다.

onUpdated | 상태 변경 감지

src/components/TimerBox.vue

```
// 반응형 데이터가 변경되어 화면이 다시 그려진 후 호출
onUpdated(() => {
  // 1. 변경된 값 로깅 (디버깅)
  console.log(
    `TimerBox onUpdated: seconds = ${seconds.value}`
  )

  // 2. DOM 기반 후처리 예시
  // if (chartInstance) chartInstance.update()
})
```

TRIGGER POINT

DOM 갱신 완료 직후

반응형 상태(State/Props)가 변경되어 Virtual DOM 리렌더링과 패치가 완료된 순간 호출됩니다.

USE CASES

주요 활용 사례

단순한 데이터 감시는 watch가 더 적합하지만, 다음과 같은 경우 필수적입니다.

외부 차트 라이브러리 데이터 갱신

콘텐츠 변경 후 스크롤 위치 조정



Infinite Loop 주의

이 훅 내부에서 반응형 상태를 다시 변경하면, 또 다시 업데이트가 발생하여 무한 루프에 빠질 수 있습니다.

onUnmounted | 타이머 정리 (Clean-up)

src/components/TimerBox.vue

```
// 컴포넌트가 화면에서 사라지기 직전에 실행
onUnmounted(() => {
  console.log('TimerBox onUnmounted: 타이머 정리' )

  // 저장해 둔 intervalId로 타이머 중지
  if(intervalId !== null) {
    clearInterval(intervalId)
  }
})
```

CRITICAL STEP

자원 해제의 필수 관문

컴포넌트가 DOM에서 제거되기 전에 호출됩니다.

이 hooks를 작성하지 않으면, 화면에서 컴포넌트가 사라져도 타이머는 뒷단에서 계속 돌아가게 됩니다.

LOGIC FLOW

메모리 누수 방지

SPA(Single Page Application) 특성상 페이지 이동만으로는 자바스크립트 상태가 초기화되지 않습니다.

setInterval / setTimeout 중지

window.addEventListener 제거

WebSocket 연결 종료 등

! 왜 중요할까요?

정리하지 않은 타이머가 쌓이면 메모리 누수(Memory Leak)가 발생하여 앱 성능이 점점 느려지거나 예기치 않은 동작을 유발합니다.

STEP-BY-STEP

실습 가이드 | 콘솔 흐름 확인

브라우저 개발자 도구(F12)를 열고 아래 순서대로 진행해 보세요.

1 콘솔 창 열기 & 준비

Chrome: F12 또는 Cmd+Opt+I를 눌러 'Console' 탭을 확인합니다.

2 '타이머 보이기' 버튼 클릭

TimerBox 컴포넌트가 마운트되면서 onMounted 로그가 찍히고 타이머가 시작됩니다.

✓ 확인: "TimerBox onMounted: 타이머 시작"

3 1초마다 자동 업데이트 관찰

숫자가 1씩 증가할 때마다 onUpdated 혹은 호출되는 것을 확인합니다.

🔄 확인: "seconds = 1, 2, 3..."

4 '타이머 숨기기' 버튼 클릭

컴포넌트가 사라지며 onUnmounted가 호출되고, 더 이상 로그가 찍히지 않아야 합니다.

🗑 확인: "TimerBox onUnmounted: 타이머 정리"

CONSOLE OUTPUT PREVIEW

● Live

```
10:00:00 > TimerBox onMounted: 타이머 시작
10:00:01 > TimerBox onUpdated: seconds = 1
10:00:02 > TimerBox onUpdated: seconds = 2
10:00:03 > TimerBox onUpdated: seconds = 3
10:00:04 > TimerBox onUnmounted: 타이머 정리

// 더 이상 로그가 발생하지 않음
```

마무리 & 확장 아이디어

Vue.js 라이프사이클 훅 학습을 마치며



생성 단계

반응형 데이터 &
함수 준비



onMounted

마운트 단계

초기화 로직
DOM 접근 / API 호출



onUpdated

업데이트 단계

변경 감지 후
차트/UI 후처리



onUnmounted

언마운트 단계

타이머/리스너
정리 (Clean-up)

더 알아볼 Hook (Advanced)

<https://ko.vuejs.org/api/composition-api-lifecycle.html>